LAPORAN
WORKSHOP SISTEM INFORMASI WEB SERVER
(TIF330805)
SEMESTER III



TUGAS PRAKTIKUM ACARA 20

NAMA:

RAJATA PRINGGIT BINTANG BIHAR

GOLONGAN:

A

NIM:

E41250264

PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)

TEKNIK INFORMATIKA KAB. NGANJUK

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI JEMBER

TAHUN 2026

PENDAHULUAN

1. Latar Belakang

Pada Acara 19 sudah tersedia tabel users dan endpoint register, tetapi seluruh endpoint mahasiswa masih bisa diakses siapa saja. Acara 20 melengkapinya dengan autentikasi berbasis token: pengguna melakukan login, menerima JSON Web Token (JWT), lalu menyertakannya pada header Authorization saat mengakses endpoint yang dilindungi. Acara ini juga menerapkan penanganan kesalahan (error handling) agar kegagalan di server tidak membocorkan detail internal ke client.

2. Tujuan

1. Membuat class Auth pada lapisan Core untuk membuat dan memvalidasi JWT (HS256) tanpa pustaka pihak ketiga.

2. Menambahkan aksi login() pada AuthController dan mendaftarkannya pada Router.

3. Melindungi method store(), update(), dan destroy() pada MahasiswaController dengan Auth::requireAuth().

4. Menerapkan try-catch pada Model untuk menangani kesalahan database dengan kode 500.

5. Menguji alur login, akses dengan token, tanpa token, dan token tidak sah menggunakan Postman.

6. Mengerjakan tugas mandiri: diskusi logout pada JWT stateless dan masa berlaku token 5 menit.

3. Dasar Teori

JSON Web Token (JWT) menurut RFC 7519 adalah token ringkas berisi tiga bagian yang dipisahkan titik (Tabel 1). Karena stateless, server tidak menyimpan sesi login; cukup memverifikasi signature token pada setiap request. Pada PHP Native, signature dibuat dengan hash_hmac() (HMAC-SHA256) dan Base64URL dari base64_encode(), sehingga tidak perlu Composer atau pustaka eksternal.

Tabel 1. Bagian-bagian JWT

Bagian	**Isi**
Header	Tipe token dan algoritma, misalnya {"alg":"HS256","typ":"JWT"}.
Payload	Klaim data pengguna (id, role) dan exp, yaitu waktu kedaluwarsa dalam detik Unix.
Signature	HMAC-SHA256 dari "header.payload" dengan kunci rahasia server, untuk memastikan token tidak diubah.

Middleware autentikasi adalah lapisan perantara yang memeriksa token sebelum request diteruskan ke logika utama. Bila token tidak ada atau tidak sah, proses dihentikan dengan kode 401 (Unauthorized). Kesalahan pada RESTful API sebaiknya selalu dikembalikan dalam format JSON yang konsisten.

Kesalahan client (4xx) mencakup endpoint salah, data tidak valid, dan token tidak sah, sedangkan kesalahan server (5xx) mencakup koneksi database gagal atau kesalahan program.

HASIL PRAKTIK

1. Implementasi

a. Perubahan File

Acara 20 menambah class Auth, aksi login, dan proteksi endpoint. Model Mahasiswa hanya ditambah try-catch.

Tabel 2. Daftar perubahan file

File	Status	Keterangan
app/core/Auth.php	Baru	generateToken(), validateToken(), requireAuth()
app/controllers/AuthController.php	Diubah	Ditambah aksi login()
app/core/Router.php	Diubah	Resource login (hanya POST)
app/controllers/MahasiswaController.php	Diubah	Auth::requireAuth() pada store(), update(), destroy()
app/models/Mahasiswa.php	Diubah	try-catch PDOException (500)
public/.htaccess	Diubah	Meneruskan header Authorization ke PHP
postman/*.json	Baru	Collection dengan token otomatis dan environment

b. Langkah 1 - Class Auth pada Lapisan Core

Class Auth berisi tiga bagian: generateToken() membuat JWT dengan exp, validateToken() memeriksa signature dengan hash_equals() lalu memeriksa exp, dan requireAuth() berfungsi sebagai middleware yang menjawab 401 bila token tidak ada atau tidak sah. Masa berlaku token diatur oleh konstanta TOKEN_TTL = 300 detik (tugas mandiri 2).

```php
<?php
// JWT (HS256) tanpa pustaka pihak ketiga: hash_hmac() + Base64URL.
class Auth {
    const SECRET = 'kunci_rahasia_workshop_2026';
    // Tugas Praktik Mandiri 2: masa berlaku token 5 menit (300 detik)
    const TOKEN_TTL = 300;

    private static function base64url(string $data): string {
        return rtrim(strtr(base64_encode($data), '+/', '-_'), '=');
    }

    private static function base64urlDecode(string $data): string {
        return (string) base64_decode(strtr($data, '-_', '+/'));
    }

    public static function generateToken(array $payload): string {
        $header = self::base64url(json_encode(['alg' => 'HS256', 'typ' =>
'JWT']));
        $payload['exp'] = time() + self::TOKEN_TTL;
```

```
        $payloadEnc = self::base64url(json_encode($payload));
        $signature = self::base64url(hash_hmac('sha256',
"$header.$payloadEnc", self::SECRET, true));
        return "$header.$payloadEnc.$signature";
    }

    // Mengembalikan payload (array) bila token sah, atau false bila tidak
sah/kedaluwarsa
    public static function validateToken(string $token) {
        $parts = explode('.', $token);
        if (count($parts) !== 3) return false;
        [$header, $payload, $signature] = $parts;

        $valid = self::base64url(hash_hmac('sha256', "$header.$payload",
self::SECRET, true));
        if (!hash_equals($valid, $signature)) return false;

        $data = json_decode(self::base64urlDecode($payload), true);
        if (!is_array($data) || !isset($data['exp']) || $data['exp'] < time())
return false;
        return $data;
    }

    // Membaca header Authorization (kompatibel Apache/XAMPP dan server bawaan
PHP)
    private static function getBearerToken(): ?string {
        $header = $_SERVER['HTTP_AUTHORIZATION'] ??
$_SERVER['REDIRECT_HTTP_AUTHORIZATION'] ?? '';
        if ($header === '' && function_exists('getallheaders')) {
            foreach (getallheaders() as $name => $value) {
                if (strcasecmp($name, 'Authorization') === 0) $header =
$value;
            }
        }
        return preg_match('/^Bearer\s+(\S+)$/i', trim($header), $m) ? $m[1] :
null;
    }

    // Middleware sederhana: hentikan request dengan 401 bila token tidak
ada/tidak sah
    public static function requireAuth(): array {
        $token = self::getBearerToken();
        if ($token === null) {
            Response::json(401, false, 'Token tidak ditemukan');
        }
        $payload = self::validateToken($token);
        if (!$payload) {
            Response::json(401, false, 'Token tidak valid atau kedaluwarsa');
```

```
        }
        return $payload;
    }
}
```

c. Langkah 2 - Aksi login() pada AuthController

login() memeriksa kelengkapan input (400), mencari pengguna, lalu mencocokkan kata sandi dengan password_verify(). Pesan 401 dibuat sama untuk username salah maupun password salah agar tidak memberi tahu penyerang akun mana yang ada. Bila berhasil, dikembalikan token dan role.

```
    public function login(): void {
        $data = json_decode(file_get_contents('php://input'), true);

        $errors = [];
        foreach (['username', 'password'] as $field) {
            if (!is_array($data) || empty($data[$field]) ||
!is_string($data[$field])) {
                $errors[$field] = "Field $field wajib diisi";
            }
        }
        if ($errors) {
            Response::json(400, false, 'Username dan password wajib diisi',
null, $errors);
        }

        $user = $this->model->findByUsername($data['username']);
        // Pesan sama untuk username salah maupun password salah (tidak
membocorkan akun mana yang ada)
        if (!$user || !password_verify($data['password'], $user['password']))
{
            Response::json(401, false, 'Username atau password salah');
        }

        $token = Auth::generateToken(['id' => $user['id'], 'role' =>
$user['role']]);
        Response::json(200, true, 'Login berhasil', ['token' => $token, 'role'
=> $user['role']]);
    }
}
```

d. Langkah 3 - Resource login pada Router

Resource login didaftarkan pada peta Router dan ditangani bersama register sebelum blok switch. Request selain POST dijawab 405.

```
    private array $map = [
        'mahasiswa' => 'MahasiswaController',
        'buku'      => 'BukuController',
        'register'  => 'AuthController',
        'login'     => 'AuthController',
    ];
...

   if ($resource === 'login' || $resource === 'register') {
            if ($_SERVER['REQUEST_METHOD'] !== 'POST') {
                Response::json(405, false, 'Method tidak diizinkan');
            }
            $controller->$resource();
            return;
        }
```

e. Langkah 4 - Melindungi Method Controller

Auth::requireAuth() dipanggil pada baris pertama store(), update(), dan destroy(), sedangkan index() (GET) tetap publik. Karena Apache pada XAMPP kadang tidak meneruskan header Authorization ke PHP, ditambahkan dua baris RewriteRule pada public/.htaccess.

```
   public function store(): void {
        Auth::requireAuth();
        $data = json_decode(file_get_contents('php://input'), true);

   public function update($id): void {
        Auth::requireAuth();
        $data = json_decode(file_get_contents('php://input'), true);

   public function destroy($id): void {
        Auth::requireAuth();
```

```
# public/.htaccess
RewriteCond %{HTTP:Authorization} .
RewriteRule .* - [E=HTTP_AUTHORIZATION:%{HTTP:Authorization}]
```

f. Langkah 5 - Try-Catch pada Lapisan Model

Operasi database dibungkus try-catch. Bila PDOException terjadi, client hanya menerima pesan umum dengan kode 500, sedangkan detail kesalahan database tidak dikirim keluar.

```
    public function create(array $data): string {
        try {
            if ($this->existsByNim($data['nim'])) {
                throw new DomainException('NIM sudah terdaftar');
            }
            $stmt = $this->db->prepare(
                'INSERT INTO mahasiswa (nim, nama, prodi) VALUES (:nim, :nama,
:prodi)'
            );
            $stmt->execute([':nim' => $data['nim'], ':nama' => $data['nama'],
':prodi' =>
                $data['prodi']]);
            return $this->db->lastInsertId();
        } catch (PDOException $e) {
            // Detail error database tidak dikirim ke client
            Response::json(500, false, 'Terjadi kesalahan pada server');
        }
    }
```

g. Langkah 6 - Pengujian Alur Autentikasi

Pada Postman, Collection diberi Authorization tipe Bearer Token dengan nilai {{token}}. Request Login sukses memiliki skrip Tests yang menyimpan token ke variabel {{token}} secara otomatis, sehingga request lain yang memakai Authorization dari Collection langsung membawa token terbaru. Request tanpa token memakai pengaturan No Auth, dan request token diubah memakai {{token}}x.

2. Tugas Praktik Mandiri

a. Mengapa JWT stateless tidak memiliki logout di sisi server

Pada JWT, server tidak menyimpan apa pun tentang sesi: keabsahan token hanya ditentukan oleh signature dan waktu exp. Akibatnya tidak ada "sesi" di server yang bisa dihapus saat pengguna logout. Token yang sudah diterbitkan tetap sah sampai exp, siapa pun yang memegangnya (misalnya token yang bocor) tetap bisa dipakai. Logout biasa hanya berarti client membuang token dari penyimpanannya.

Agar token bisa dicabut sebelum kedaluwarsa dibutuhkan mekanisme tambahan, yaitu blacklist token (tabel berisi token yang dicabut dan diperiksa pada setiap request), masa berlaku token yang singkat disertai refresh token, atau mengganti kunci rahasia (yang membatalkan semua token sekaligus). Blacklist mengurangi sifat stateless karena server harus menyimpan data lagi. Kesimpulannya, JWT lebih ringan dan mudah diskalakan, tetapi pencabutan token menjadi lebih sulit dibanding sesi di server. Karena itu masa berlaku token sebaiknya dibuat pendek.

b. Masa berlaku token 5 menit

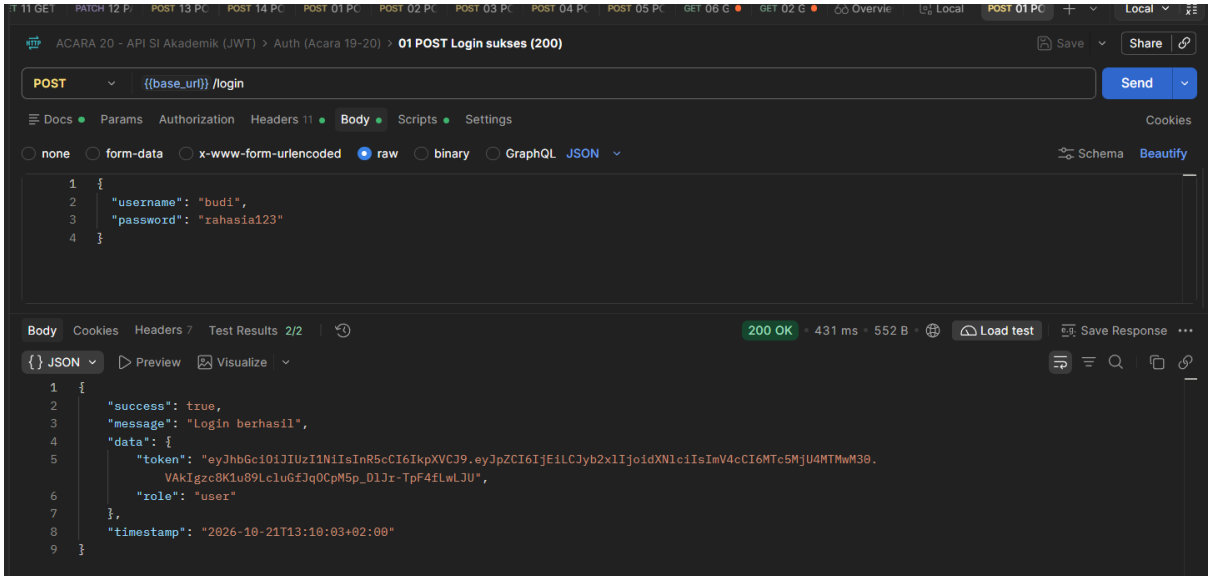
Pada Auth::generateToken(), exp diubah dari time() + 3600 menjadi time() + TOKEN_TTL dengan TOKEN_TTL = 300 detik. Setelah login, token dipakai untuk POST /mahasiswa (berhasil, 201). Bila request yang sama dikirim lagi setelah lebih dari 5 menit tanpa login ulang, server menjawab 401 "Token tidak valid atau kedaluwarsa" (Gambar 13). Setelah login ulang, token baru berlaku kembali selama 5 menit.

3. Pengujian API

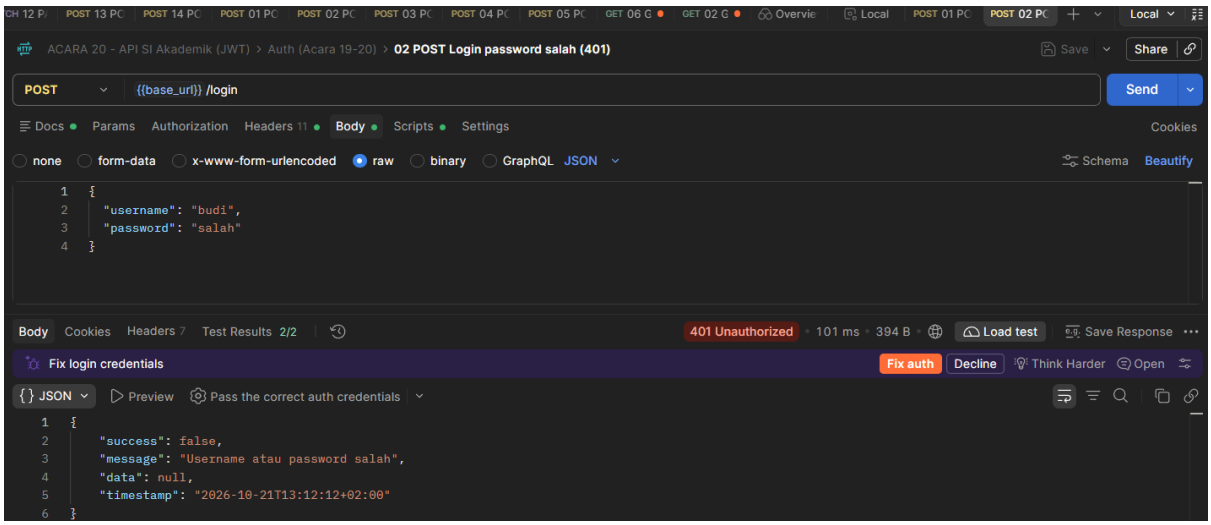
Pengujian dilakukan memakai Postman dengan Environment Local. Urutan request mengikuti Tabel 3 karena request 06 sampai 12 memakai token dari request 01 dan data mahasiswa dari request 08.

Tabel 3. Hasil pengujian

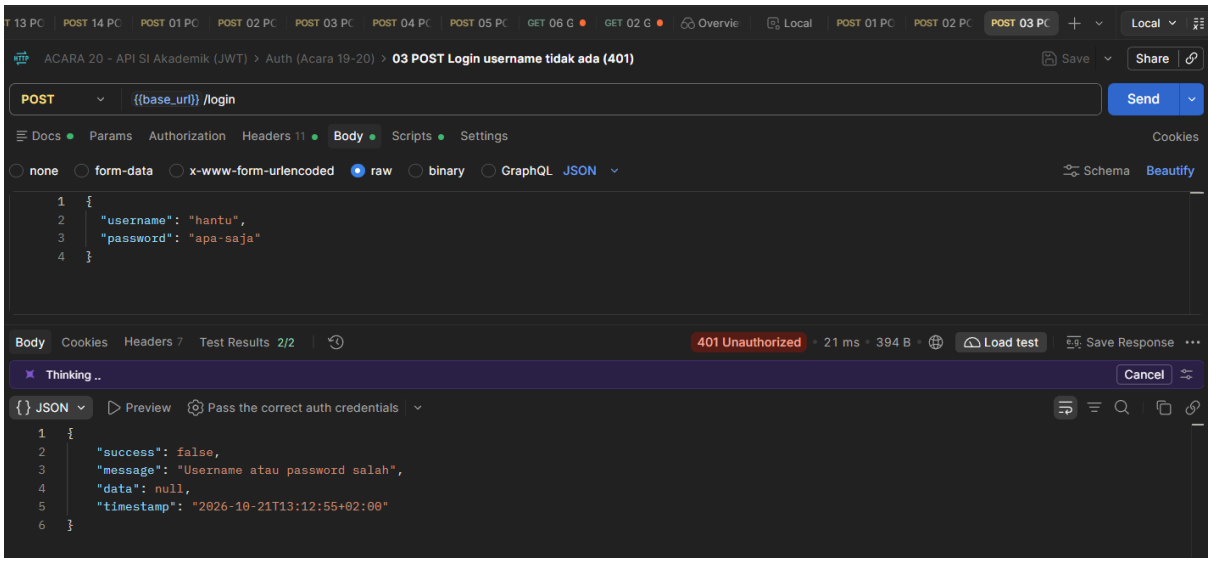
No	Skenario	Request	Status	Gambar
1	Login sukses	POST /login	200 OK	1
2	Password salah	POST /login	401 Unauthorized	2
3	Username tidak ada	POST /login	401 Unauthorized	3
4	Field kosong	POST /login	400 Bad Request	4
5	Method salah	GET /login	405 Method Not Allowed	5
6	Tambah data tanpa token	POST /mahasiswa	401 Unauthorized	6
7	Tambah data token diubah	POST /mahasiswa	401 Unauthorized	7
8	Tambah data dengan token	POST /mahasiswa	201 Created	8
9	Ubah data dengan token	PUT /mahasiswa?id=1	200 OK	9
10	GET tanpa token (publik)	GET /mahasiswa	200 OK	10
11	Hapus data tanpa token	DELETE /mahasiswa?id=1	401 Unauthorized	11
12	Hapus data dengan token	DELETE /mahasiswa?id=1	200 OK	12
13	Token kedaluwarsa (setelah 5 menit)	POST /mahasiswa	401 Unauthorized	13
14	Kesalahan database (tabel tidak ada)	POST /mahasiswa	500 Internal Server Error	14



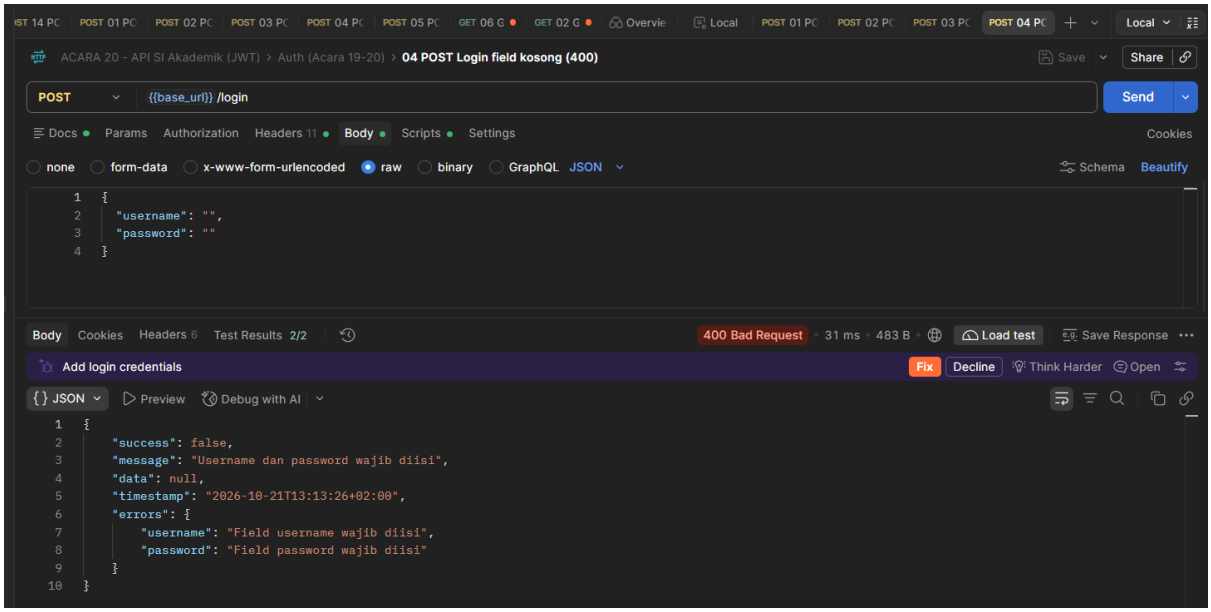
Gambar 1. POST /login berhasil: 200 OK disertai token



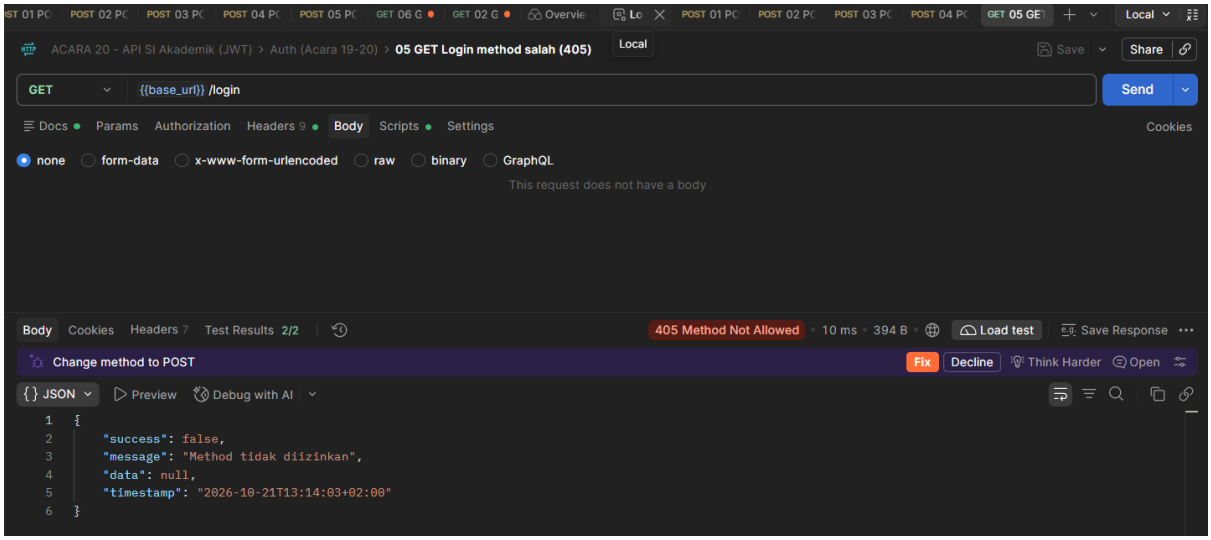
Gambar 2. POST /login password salah: 401 Unauthorized



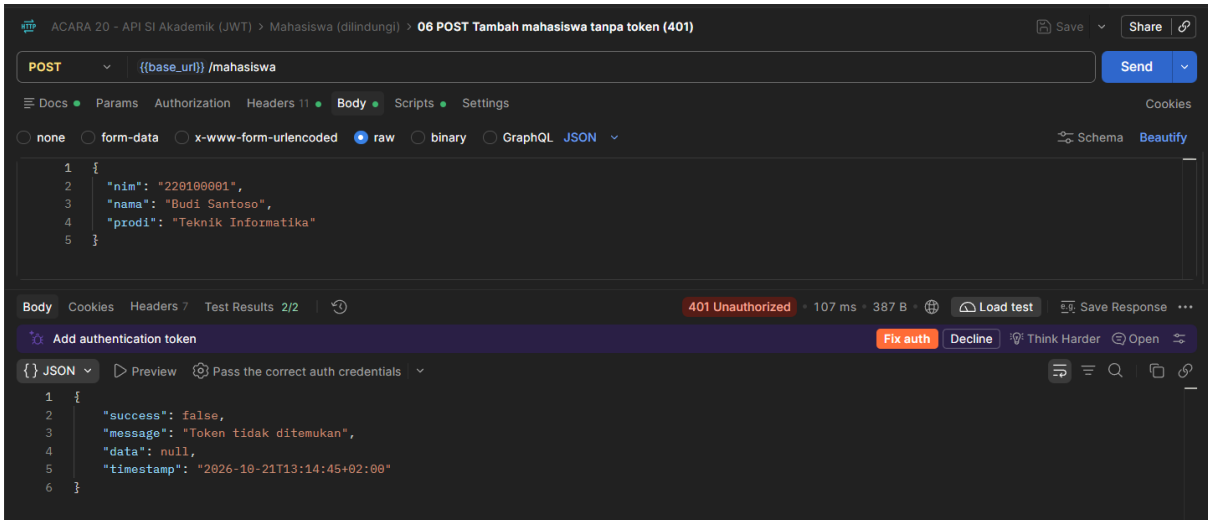
Gambar 3. POST /login username tidak ada: 401 Unauthorized



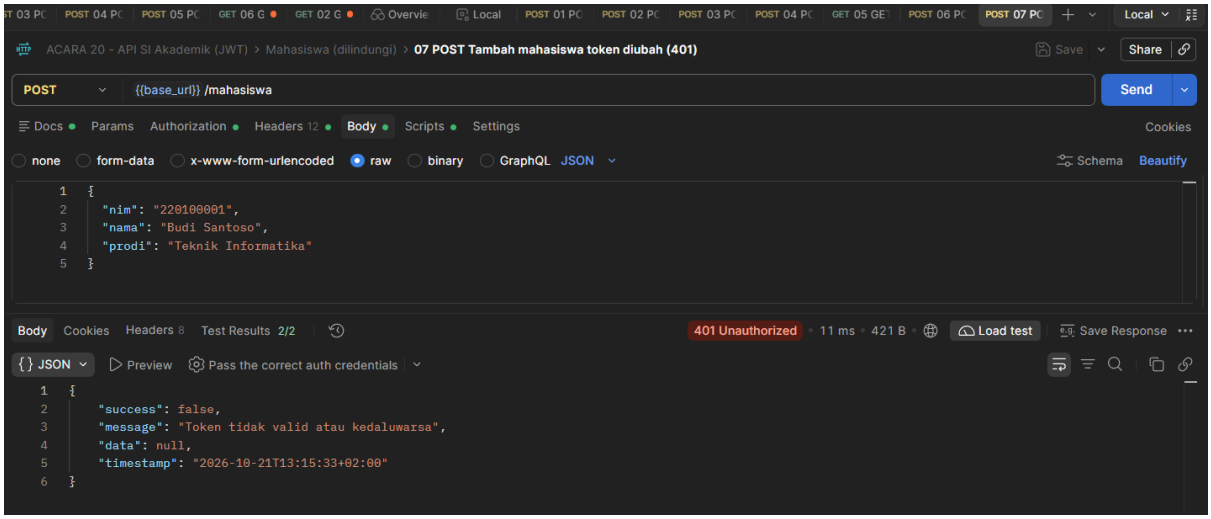
Gambar 4. POST /login field kosong: 400 Bad Request



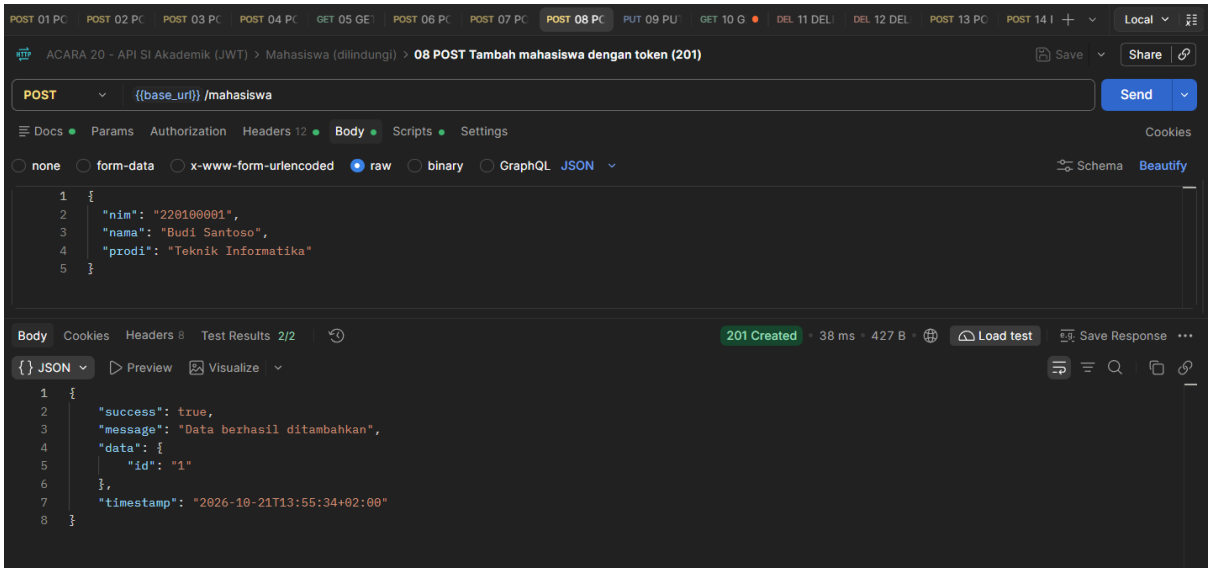
Gambar 5. GET /login: 405 Method Not Allowed



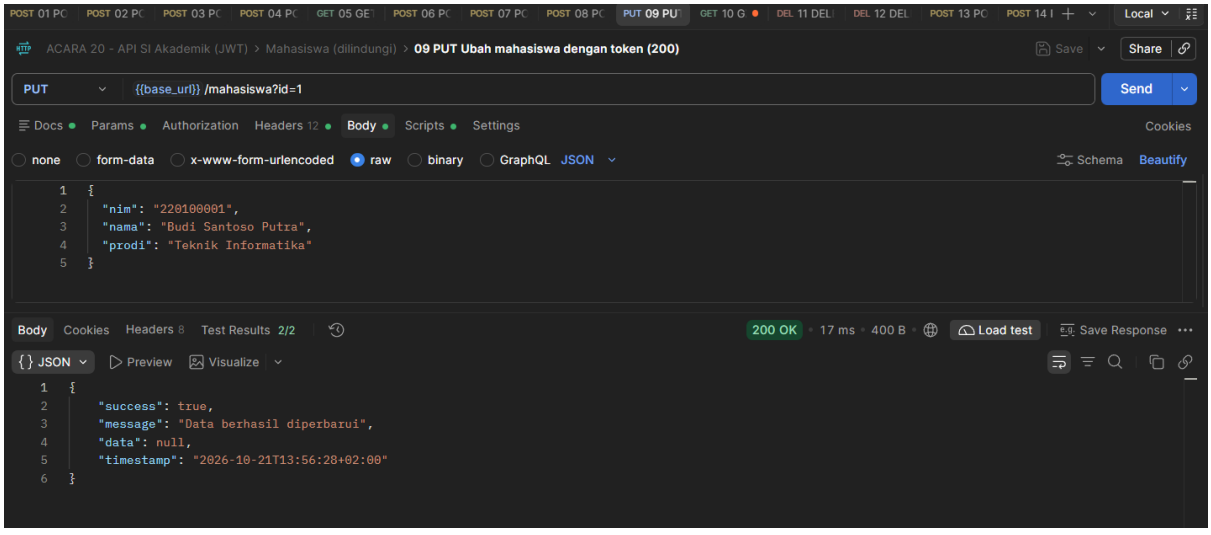
Gambar 6. POST /mahasiswa tanpa token: 401 Unauthorized



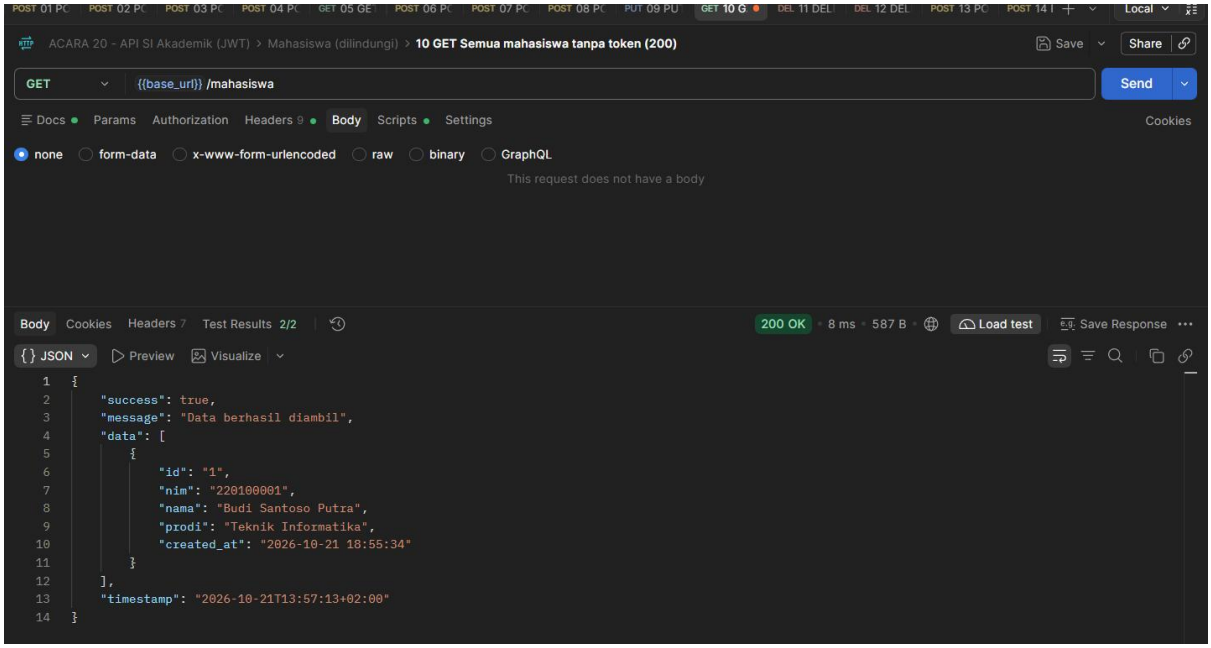
Gambar 7. POST /mahasiswa dengan token diubah: 401 Unauthorized



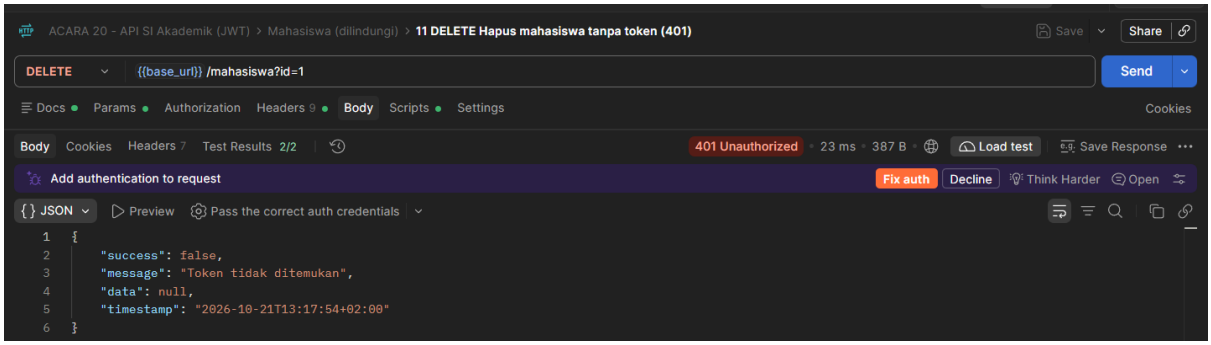
Gambar 8. POST /mahasiswa dengan token: 201 Created



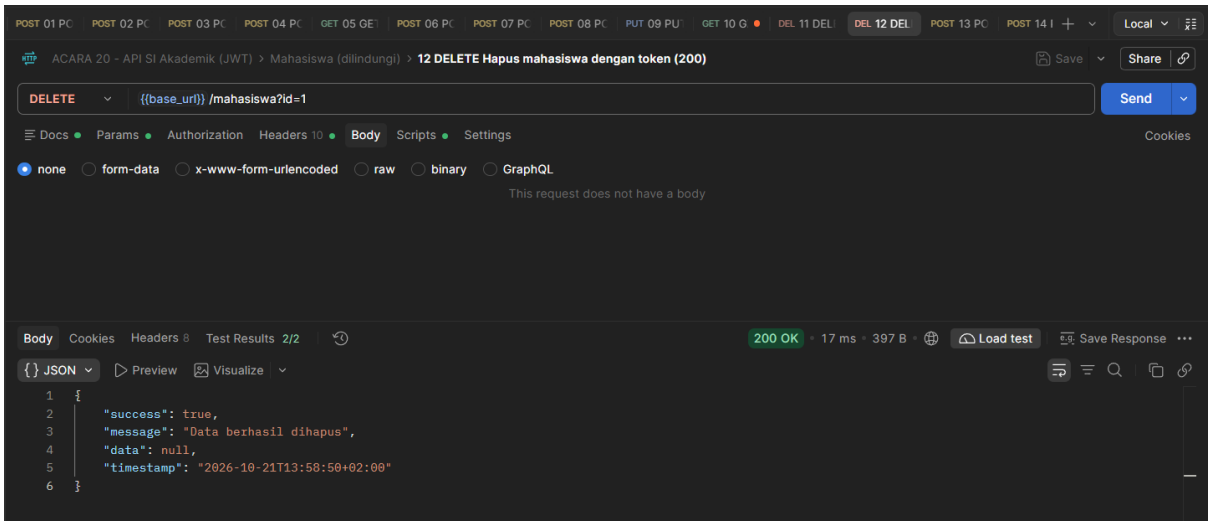
Gambar 9. PUT /mahasiswa?id=1 dengan token: 200 OK



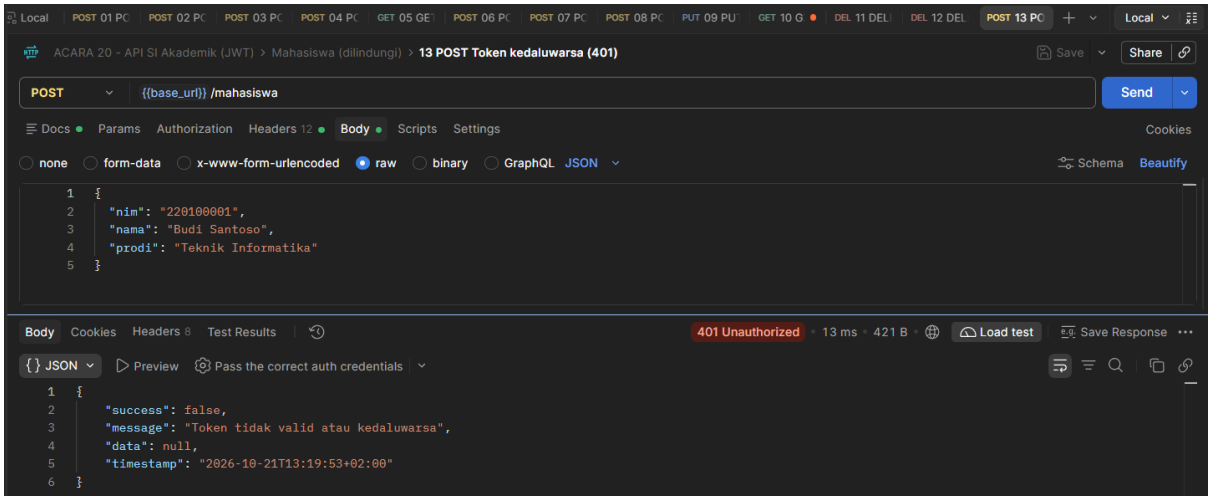
Gambar 10. GET /mahasiswa tanpa token tetap 200 OK



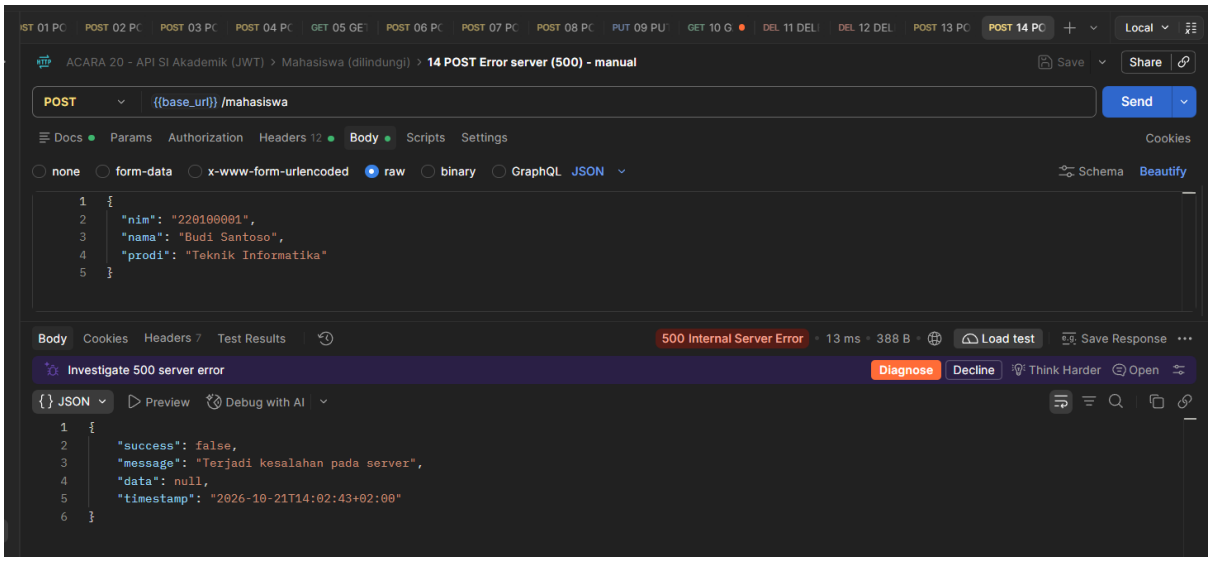
Gambar 11. DELETE /mahasiswa?id=1 tanpa token: 401 Unauthorized



Gambar 12. DELETE /mahasiswa?id=1 dengan token: 200 OK



Gambar 13. Token kedaluwarsa setelah 5 menit: 401 Unauthorized



Gambar 14. Kesalahan database: 500 dengan pesan umum

4. Pembahasan

Alur autentikasi berjalan sesuai rancangan: login yang benar menghasilkan token, endpoint tulis (POST, PUT, DELETE) menolak request tanpa token atau dengan token yang diubah dengan kode 401, dan GET tetap publik. Token yang diubah satu karakter saja ditolak karena signature HMAC tidak lagi cocok. Pesan error login dibuat sama untuk username maupun password yang salah supaya tidak membocorkan akun yang terdaftar. Pada kesalahan database, client hanya menerima pesan umum dengan kode 500 sehingga struktur tabel dan detail query tidak terbuka.

Beberapa penyesuaian terhadap kode modul dilakukan agar lebih andal. Pertama, payload token didekode dengan Base64URL yang benar (mengembalikan karakter - dan _ menjadi + dan /), karena base64_decode() langsung dapat gagal pada payload yang mengandung kedua karakter itu. Kedua, header Authorization dibaca dari $_SERVER dengan cadangan getallheaders() tanpa membedakan huruf besar kecil, ditambah aturan pada .htaccess untuk Apache. Ketiga, login memeriksa input kosong lebih dulu (400). Keempat, try-catch dipasang pada create(), update(), dan delete() sekaligus, bukan hanya create().

Catatan keamanan: kunci rahasia SECRET masih tertulis di kode sehingga tidak boleh ikut di-commit ke repositori publik, dan pada deployment sebaiknya dipindahkan ke konfigurasi lingkungan serta dikombinasikan dengan HTTPS. Pemanggilan Response::json() dari Model (untuk kode 500) menyatukan dua lapisan; pada aplikasi yang lebih besar, Model sebaiknya melempar exception dan Controller yang menerjemahkannya menjadi response.

KESIMPULAN

1. Class Auth berhasil dibuat untuk menerbitkan dan memvalidasi JWT HS256 tanpa pustaka pihak ketiga, dan aksi login() menghasilkan token setelah kata sandi diverifikasi dengan password_verify().
2. Endpoint POST, PUT, dan DELETE mahasiswa terlindungi: tanpa token atau dengan token yang diubah atau kedaluwarsa dijawab 401, sedangkan GET tetap publik.
3. Error handling pada Model berhasil: kesalahan database menghasilkan 500 dengan pesan umum tanpa membocorkan detail.
4. Tugas mandiri selesai: JWT stateless tidak punya logout di server tanpa mekanisme tambahan seperti blacklist, dan masa berlaku token 5 menit teramati dengan respons 401 setelah token kedaluwarsa.

DAFTAR PUSTAKA

Pratama, R. A., & Hasanah, Q. (2026). Buku Kerja Praktik Mahasiswa (BKPM) Workshop Sistem Informasi Web Server (TIF330805). Program Studi D4 Teknik Informatika PSDKU Nganjuk, Jurusan Teknologi Informasi, Politeknik Negeri Jember.